

QLS Project — Comprehensive Handoff Document

Full-detail snapshot. Paste into a new chat or upload to project knowledge to resume seamlessly.

0. The Main Goal (one sentence)

To investigate whether **adaptive control of neighbourhood selection** improves Quantum Local Search (QLS) for Max-Cut — specifically whether it helps classical local search **escape one-flip local optima** more effectively than fixed hybrid QLS and strong classical baselines.

Every artefact in the project is a different approach to this one goal: *help classical local search get out of local minima more reliably, more efficiently, or more effectively.*

1. The Three Core Claims (project stands or falls on these)

1. **Selector matters.** The frustrated selector ($|\text{flip-gain}| \approx 0$), connected-frustrated, or another principled selector produces measurably better local-optimum escape than random selection (current QLS default).
2. **Adaptive triggering helps.** EMA-based trigger + adaptive-k update beat fixed QLS.
3. **Improvements are real.** Gains hold against strong classical baselines — tabu search, Breakout Local Search (BLS, Benlic & Hao 2013), not just naive hill climbing.

Everything else (cascades, meta-selectors, GPU/LLM/QAOA backends) is extension that strengthens the contribution and protects against null results.

2. The Paradigm-Agnostic Reframing

The project is really a **paradigm-agnostic adaptive local subproblem solver** with a swappable backend. This framing survives a null result on quantum — the question becomes "*which computing paradigm is most effective as a local neighbourhood solver, and does adaptive selection of the neighbourhood make any of them work better?*"

3. The Full 52-Item Project Map

Core algorithms

1. One-flip local search (shared engine)
2. QUBO subproblem builder
3. QLS — baseline, random selector, fixed k
4. Adaptive QLS — adaptive trigger, pluggable selector, adaptive k

Neighbourhood selectors (6 total — pluggable into Adaptive QLS)

5. **Random** — Tomesh et al. 2022 baseline (uniform sample of k vertices)
6. **Frustrated** — top- k by $|\text{gain}|$ ascending (novel; $|\text{gain}| \approx 0$ = indifferent vertices)
7. **Frustrated-Connected** — seed from most-frustrated vertex, grow connected subgraph greedily by lowest $|\text{gain}|$ from frontier (novel)
8. **Impact** — top- k by $|\text{gain}|$ descending (qbsolv / Atobe 2022)
9. **Clustering** — spectral clustering of solution-pool correlations (Zhao & Tang 2025)
10. **Meta-rule** — rule-based switching between selectors

Meta-selector levels (Levels 2/3 not implemented)

11. Level 2 — online logistic regression
12. Level 3 — contextual bandit (LinUCB / Thompson Sampling)

Adaptive mechanisms

13. **EMA trigger** — replaces fixed `stag_thresh`; `ema_esc` = 0.5 init, $\alpha=0.3$, threshold 0.2
14. **Momentum-damped k update** — 10-call window, 3 sub-windows of 3, consistent-trend detection, 5-call cooldown, ± 2 grow/shrink
15. **Degree-weighted tiebreaker** — for plateau collapse when `frac_zero` > 0.8
16. **Pool persistence on restart** — keep top-5 incumbents (triggered when `ema_esc` < 0.05)
17. **Pool warm-start** — 5 diversified cuts before main loop
18. **Pool size guard** — fallback to frustrated if pool too small for clustering
19. **Cluster caching** — recompute only when pool changes significantly
20. **GainCache** — explicit `invalidate()` / `assert_valid()` (fixes silent stale-gain bug)

Backends (8 paradigms)

21. **neal** — D-Wave simulated annealing — Phase 1 
22. **OpenJij SQA** — simulated quantum annealing — Phase 1 
23. **Exact brute-force** — $|S| \leq 20$, ablation ground truth — Phase 1 
24. **Simulated Bifurcation (SBM)** — OpenJij — Phase 2 (30 min effort)
25. **GPU parallel SA** — PyTorch 1000 chains — Phase 2 (1 day)
26. **QAOA** — PennyLane gate-model — Phase 2 (2 days)
27. **GNN solver** — PyTorch Geometric — Phase 3 (4 days, Nice-to-have)
28. **LLM solver** — Claude API — Phase 3 (2 hours, Nice-to-have)
29. **Neuromorphic** — Intel Lava SDK — Phase 4 (Optional)

Multi-paradigm cascade architectures

30. **Sequential cascade** — SA $\rightarrow$ SBM $\rightarrow$ exact, escalate on failure (1h effort)

- 31. **Warm-chain relay** — SBM output → QA reverse anneal → SA polish (2h effort)
- 32. **Parallel portfolio** — all backends simultaneously, take best (half day)

Classical baselines

- 33. Simulated annealing (improved) ✓
- 34. Tabu search ✓
- 35. Breakout Local Search (Benlic & Hao 2013) — not yet implemented

Evaluation & infrastructure

- 36. Equal wall-clock budget comparison
- 37. Equal call-count budget comparison
- 38. 30 independent trials per configuration
- 39. Median ± IQR reporting
- 40. Wilcoxon signed-rank test (non-parametric)
- 41. **Outcome metrics:** cut value, approximation ratio, time-to-target
- 42. **Process metrics (novel):** escape rate, mean delta per call, k trace, paradigm trace, plateau fraction
- 43. Ablation studies — same controller, swap backends
- 44. Stratified instance testing — small ($n \leq 800$) for selector comparison
- 45. Scaled k testing — $k = \lceil 0.05n \rceil$ alongside fixed k
- 46. **QUBO sign unit test** — K4 known optimum (critical: sign convention bug catches here)
- 47. Runtime sanity check — merged proposal never worse than any one-flip from x
- 48. Plateau detection metric — frac_zero logged separately
- 49. **Paradigm escape contribution metric** — which backend produced each escape (novel)

Benchmark instances

- 50. G-set: G1-G22 ($n=800$), G43-G54 ($n=1000$), up to G81 ($n=20000$)
- 51. Random 3-regular graphs
- 52. Erdős-Rényi, SBM/community graphs

4. Key Hyperparameters & Constants

G-set best-known values (used in code)

```
python
```

```
best_known_cuts = {
    'G1': 11624, 'G11': 564, 'G14': 3064, 'G22': 13359
}
```

Adaptive QLS defaults

- `k_min=5`, `k_max=30` (default range)
- `n_reads=100` (general), `n_reads=200` (k-sweep experiments)
- `ema_esc` init = 0.5, `alpha` = 0.3, trigger threshold 0.2
- Pool max size = 20; top-5 kept on persistence
- Pool warm-start = 5 diversified cuts
- Plateau detection: `eps = 1e-6`, `frac_zero > 0.8` triggers degree tiebreaker
- k window = 10 calls, sub-windows of 3, cooldown = 5 calls, step = ± 2

Improved SA hyperparameters

- `chi0 = 0.80` (Ben-Ameur target initial acceptance probability)
- `beta_final_multiplier = 5.0` ($\beta_{\text{final}} = \beta_{\text{initial}} \times 5$)
- `reheat_threshold = 10` (reheat after $10 \times N$ non-improving flips)
- `max_reheats = 3` per run
- Vertex selection: 70% top-10% by gain, 30% random
- Schedule: linear-in- β (Myklebust 2015) — NOT geometric
- T_{initial} calibration: 200 samples, $T = -\text{mean_bad} / \ln(\text{chi0})$, floor 0.01

Experiment budgets (current)

- 30 seconds per trial
- 10 trials (initial), 30 trials (final)

G-set parameter recipes (from literature, for reference)

Source	β_{initial}	β_{final}	Schedule	Outcome
Myklebust 2015 (BKS holder)	0	10000/best	linear-in- β	G11 = 564 (BKS)
Haribara/Wang 2016	implicit	implicit	logarithmic	G1 best 11624
Isakov 2015	0.1 ($\pm J$)	3.0 ($\pm J$)	linear-in- β	(Chimera)
D-Wave Neal default	~ 0.014 (G1)	~ 4.6	geometric	95-97% BKS

For G1 dense, Ben-Ameur predicts $T_{\text{initial}} \approx 250\text{--}300$ ($\text{chi0}=0.8$, max gain ~ 60). For G11 sparse 4-reg, $T_{\text{initial}} \approx 15\text{--}20$ (max gain 4). **Our previous fixed $T_{\text{initial}} = 2.0$ was too cold for G1**

(acceptance $\approx 10^{-13}$).

5. Empirical Results To Date

G11 + G1 initial experiment (approximation ratios)

Instance	SA	Tabu	QLS-Random	AQLS-Frustrated	AQLS-Impact
G11 (sparse 4-reg)	0.991	0.867	0.819	0.782	0.819
G1 (dense deg-48)	0.994	0.997	0.984	0.980	0.982

Key findings:

- Frustrated selector: **46-48× higher escape rate** than random (0.147 vs 0.003) — but **worse cut quality** (escapes land in worse basins).
- Plain frustrated picks scattered disconnected vertices → QUBO subproblem has only **4 internal edges** vs **11** for connected frustrated (**2.75× more**).
- SA cooling schedule changes: **no significant difference** in cut quality ($p = 0.313$).
- **The acceptance rule is NOT the main problem — the neighbourhood structure IS.**
- Tabu dominates on dense G1 (reaches optimum); SA dominates on sparse G11.

Connected frustrated selector (G11 confirmation)

- Matches QLS-Random and Impact on cut quality ($p > 0.15$ — not significant)
- **Beats plain frustrated significantly** ($p = 0.004-0.008$)
- Confirms disconnected-subgraph hypothesis

k-sweep prediction (theory)

Expected pattern on G11:

- $k=10 \rightarrow$ median ~ 442 (within funnel, bad)
- $k=20 \rightarrow \sim 445$ (still within funnel)
- $k=40 \rightarrow \sim 450$ (approaching threshold)
- $k=80 \rightarrow \sim 455$ (cross-funnel confirmed empirically)
- $k=160 \rightarrow \sim 455$ (plateau, no further gain)

Key empirical claim (validated): at small k (10-40) selector choice matters enormously; at $k \approx 80$ all selectors converge — funnel crossing requires large perturbation regardless of selection strategy. Consistent with RSB ultrametric theory, Overlap Gap Property, and spectral gap predictions simultaneously.

Test suite

- 17 unit tests, all passing
 - Critical tests: K4 known-optimum QUBO sign test, connected-selector connectivity test, frustrated-vs-impact different-selection test
-

6. Complete File Structure

```
src/
  gain_cache.py          # GainCache: invalidate(), assert_valid(),
incremental_update()
  local_search.py        # OneFlipLS, random_cut(), compute_cut_value()
  qubo.py                # build_local_qubo() with boundary conditions,
merge_proposal() with runtime sanity check
  graph.py              # load_gset(), generate_random_regular(),
generate_erdos_renyi(), generate_sbm(), graph_stats()
  backends.py           # backend_exact, backend_neal, backend_sqa, get_backend() –
all accept init= for warm-start
  selectors.py          # select_random, select_frustrated,
select_frustrated_connected, select_impact, select_clustering, select_meta_rule,
get_selector(); plateau_aware_rank helper
  qls.py                # QLS baseline (random selector, fixed k)
  adaptive_qls.py       # adaptive_qls(): EMA trigger, pluggable selector, adaptive
k, pool warm-start/persistence, SA cooling, lookahead acceptance
  baselines.py          # simulated_annealing() (improved), tabu_search();
_calibrate_T_initial(), _gain_weighted_flip()
  metrics.py            # Metrics: 9 process + outcome metrics,
record_cut/record_qls_call/check_time_to_target
  experiment.py         # run_experiment(), run_gset_experiment(), run_k_sweep()
tests/
  test_qubo.py          # K4 known-optimum sign test
  test_local_search.py  # all algorithm tests, selector tests, connected-selector
connectivity test, Adaptive QLS all-selectors test
data/gset/              # G1, G11, G14, G22
results/                # JSON outputs: gset_<G>.json, k_sweep_<G>.json
venv/                  # Windows venv (path: C:\Users\thotl\OneDrive\Desktop\QLS)
```

Acceptance rules (3, all in adaptive_qls.py)

- **improvement** — accept iff $\delta > 0$; restart otherwise
- **metropolis** — fixed-T Liu & Goan 2023 style; $\exp(\gamma \cdot \delta)$
- **lookahead** — run OneFlipLS on x_{proposal} before evaluating δ (novel)
- (SA cooling variant — tested, made things worse, deprecated)

7. Core Algorithm: Adaptive QLS Pseudocode

python

```

def adaptive_qls(G, budget_seconds, selector, backend,
                 k_min=5, k_max=30, n_reads=100, ...):
    # Pool warm-start: 5 diversified cuts via OneFlipLS
    pool = [...] # length 5
    x = best in pool; x_best = copy(x); best_cut = cut(x)

    # Adaptive state
    k = k_min
    ema_esc = 0.5; alpha = 0.3
    k_window = []; k_cooldown = 0

    while elapsed < budget:
        # Phase 1: classical descent
        x, gc, at_opt, _ = one_flip_ls(G, x, gc)
        current_cut = cut(x); record
        if current_cut > best_cut: update best

        # Phase 2: EMA-based trigger
        if not at_opt and ema_esc >= 0.2:
            continue

        # Phase 3: plateau detection
        frac_zero = fraction of |gain| < 1e-6

        # Phase 4: pluggable selector (fallback to random on exception)
        S = selector(G, gc, k_actual=min(k, |V|), pool=pool, rng=rng)

        # Phase 5: local solve
        Q = build_local_qubo(G, x, S)
        x_local = backend(Q, S, n_reads=n_reads)
        x_prop = merge_proposal(x, x_local, S)

        # Phase 5b: look-ahead (if acceptance == 'lookahead')
        if acceptance == 'lookahead':
            x_prop, _, _, _ = one_flip_ls(G, x_prop, GainCache())

        delta = cut(x_prop) - current_cut
        record_qls_call(delta, k_actual, paradigm, frac_zero, sel_name)

        # Phase 6: acceptance + pool update
        if delta > 0:
            x = x_prop; gc.invalidate()
            pool.append(copy(x)); trim to 20
        else:
            if ema_esc < 0.05: # restart with pool persistence
                pool = top-5(pool)

```

```

        x = random_cut(G, rng); gc.invalidate()

# Phase 7: EMA update
ema_esc = alpha * (delta > 0) + (1 - alpha) * ema_esc

# Phase 8: momentum-damped k update
k_window.append(delta); k_cooldown -= 1
if len(k_window) >= 10 and k_cooldown <= 0:
    w = k_window[-9:]
    t0, t1, t2 = mean(w[0:3]), mean(w[3:6]), mean(w[6:9])
    if t0 < t1 < t2:         k = max(k_min, k-2); cooldown=5 # improving → s
    elif t0 > t1 > t2:       k = min(k_max, k+2); cooldown=5 # worsening → g
    k_window = []

return x_best, metrics

```

8. Literature Foundation (Key Citations)

State of the Art on G-set (May 2026)

- **BLS (Benlic & Hao 2013)** — still strong on $n \leq 5000$; improved 33/71 BKS, matched 35
- **Cosm (Zick 2025, arXiv 2505.18508)** — holds BKS for $G72 = 7,008$; $G77 = 9,940$; $G81 = 14,060$
- **Population Annealing MC (Khan & Shukla 2025, arXiv 2510.21105)** — holds BKS $G63 = 27,047$
- **Quantum/SBM hardware has NOT produced a new G-set BKS.** Toshiba SBM reaches only ~99.6% of $G81$ BKS
- **PLS-completeness:** plain one-flip FLIP is PLS-complete (Schäffer & Yannakakis 1991)

QLS / Sub-QUBO prior art

- **Tomesh, Saleem, Suchara 2022 (Quantum 6, 781)** — canonical QLS; BFS-ball neighbourhood, random root selection
- **Liu & Goan 2023 (IEEE QCE; arXiv:2304.06473)** — RL-QLS; IMPALA agent; "32_pick_5", "64_pick_5", "128_pick_5" checkpoints; only baseline is uniform random
- **Liu & Goan 2022 (arXiv:2208.03283)** — LSSA predecessor; no Metropolis acceptance
- **Atobe, Tawada, Togawa 2022** — Impact-Indexing for sub-QUBO; "largest negative impact" trapped in local minima
- **Zhao & Tang 2025 (arXiv:2502.16212)** — correlation/clustering > Impact-Indexing > Certainty-Degree > random
- **Ropke & Pisinger 2006** — ALNS foundation

- **Rosenberg, Vazifteh, Woods, Haber** (Comp. Opt. Appl.) — sub-QUBO with greedy acceptance
- **Dupont et al. 2024** — light-cone selection

Empirical pattern (synthesised)

- RL-QLS > random-QLS (Liu & Goan)
- Clustering > Impact > Certainty-Degree > random (Zhao & Tang)
- Directed > random perturbation (BLS)
- Mixed portfolio > single operator (ALNS meta-analysis, Turkeš et al. EJOR 2021, 0.14% CI 0.07-0.22%)

SA Evolution & Cooling

- **Kirkpatrick, Gelatt, Vecchi 1983** — original SA
- **Lundy & Mees 1986** — slower late-cooling schedule
- **Aarts & Korst 1989** — statistical cooling
- **Johnson, Aragon, McGeoch, Schevon 1989** — SA for graph bisection / Max-Cut
- **Ben-Ameur 2004 (Comput. Optim. Appl. 29:369-385)** — automatic T_{initial} calibration; $\chi_0 \in [0.8, 0.95]$; $T_{\text{initial}} \approx (\text{max gain}) / \ln(1/\chi_0)$
- **Myklebust 2015 (arXiv:1505.03068)** — linear-in- β winning SA on G-set; G11=564 (BKS); G1=11621
- **Isakov et al. 2015 (arXiv:1401.1084)** — linear-in- β recommendation
- **Modified Lam (Swartz 1993, Boyan 1998)** — 0.44 acceptance-rate target
- **Cicirello CP 2007 & arXiv:1709.02877** — modified-Lam reference impl
- **Onizawa & Hanyu 2024** — empirically 2 reheats per 30s helps on G1/G11; 5 reheats degrade

Multi-paradigm cascade prior art (project's novelty confirmed)

- **No paper** implements warm-chain cascade as local subproblem solver in QLS/LNS for Max-Cut
- Closest prior art: Imoto 2021, Kikuchi 2023 (full-problem chains); SBQA, TESB, Kanao-Goto heated SB (single-algorithm fusions); Google SA $\leftrightarrow$ QA patent family; Egger, Marecek, Woerner 2021 (QAOA warm-start at top level)

Theoretical foundations

- **Parisi RSB / FRSB** — spin glass theory underpinning funnel structure
- **Dembo-Montanari-Sen Parisi** constant $P^* \approx 0.7632$
- **Chen-Gamarnik-Panchenko-Rahman** Overlap Gap Property (K=2 caveat)
- **Trevisan's spectral Max-Cut bound**
- **Cheeger inequality** — algebraic connectivity λ_2 and bisection
- **Fiedler vector** — second Laplacian eigenvector for partition guidance

- **Hajek 1988** — SA convergence with logarithmic cooling
- **Goemans-Williamson 1995** — $\alpha_{\text{GW}} \approx 0.87856$ SDP bound

9. Interdisciplinary Research Completed

Discipline	Status	Key takeaway
Spin glass / RSB	✓ Done	Validated by k=80 experiment; ultrametric funnel structure
Information theory	✓ Done	Frustrated selector formalised as maximum-entropy / Bethe marginal
Spectral graph theory	✓ Done	Fiedler/Cheeger predictions confirmed; predicts G1 needs smaller k than G11 (100× larger λ_2)
SA evolution (40 years)	✓ Done	Led to improved SA (Ben-Ameur, linear- β , gain-weighted, reheating)
Basin landing / funnel structure	✓ Done	Led to connected selector + look-ahead acceptance
Perturbation size & funnel crossing	✓ Done	k=40–80 threshold confirmed
Neighbourhood selection prior art	✓ Done	Frustrated selector confirmed novel
Multi-paradigm cascade prior art	✓ Done	Cascade architecture confirmed novel
SA-style acceptance in QLS	✓ Done	Liu & Goan already published fixed-T version
G-set SOTA	✓ Done	BLS, Cosm, PAMC are benchmarks
ML / RL for local-minima escape	✓ Done	L2S, GNN, contextual bandits surveyed
Neuroscience	✓ Done	Hopfield, stochastic resonance, neuromodulators, predictive coding

Still unresearched (ranked by actionability)

1. **Cut polytope theory** — most actionable; facet-defining inequalities → principled variable

selection

2. **Fiedler-augmented selector** — combine Fiedler vector with gain-based (recommended by spectral research)
 3. **Belief propagation marginals** — principled replacement for $|\text{gain}| \approx 0$ heuristic (recommended by info-theory research)
 4. **Quantum information theory** — entanglement structure for QAOA subproblem composition
 5. **Cross-graph analysis** — all experiments so far on G11; need G1 (dense), G14 (irregular), G22 (spin glass)
-

10. Current Status

Completed

- Full literature review (Chapter 2, three core claims)
- Algorithm design: all 9 methods specified in pseudocode
- Phase 1 implementation: all core components built and tested
- 17 unit tests passing
- G11 and G1 initial experiments
- Connected frustrated selector (2.75× more internal edges confirmed)
- Improved SA (auto-calibrated T, linear-in- β , gain-weighted, reheating)
- k-sweep function written and executed

In Progress / Next

1. Analyse k-sweep results
2. Run G1 with $k=80$ (predict smaller k needed due to higher λ_2)
3. Run 30 trials final on G1, G11, G14, G22
4. Phase 2 backends: SBM (30 min), GPU SA (1 day), QAOA (2 days)
5. Sequential cascade (1h), warm-chain relay (2h)
6. Write results & analysis chapter

Key decision points

Condition	Action
Connected selector beats plain frustrated on cut quality ($p < 0.05$)	Confirm connected-selection hypothesis — key finding
Connected selector still worse than QLS-Random	Selector alone insufficient — need better acceptance or larger k

Condition	Action
Improved SA closes gap with Tabu on G11	SA tuning is the bottleneck — publish improved SA as contribution
AQLS variant matches SA on cut quality	Core research question answered positively — proceed to Phase 2
All AQLS variants significantly worse than SA	Frame as informative null result — report process metrics as contribution

Experimental bar

Result	Meaning	Verdict
Matches BLS on G1-G54	Competitive with 2013 SOTA	Acceptable
Matches BLS faster on G55-G81	Speed competitive with SBM/Cosm	Good — publishable
Beats BLS on G1-G54	New best on well-studied instances	Strong — publishable
New BKS on G63/G72/G77/G81	Genuinely new SOTA	Exceptional
First escape-rate measurement on G-set	Novel process metric regardless of cut quality	Novel — always publishable

11. Implementation Traps (known, mitigated)

- QUBO sign convention** — minimising Q vs maximising Max-Cut. One wrong sign silently inverts objective. *Mitigation:* K4 known-optimum unit test in `tests/test_qubo.py`.
- Stale GainCache after accept** — if proposal accepted, gain array from previous OneFlipLS is stale. *Mitigation:* `gc.invalidate()` after every accept; `assert_valid()` called by every selector.
- Disconnected frustrated S** — at local optimum every vertex has gain ≤ 0 , so $|gain| \approx 0$ picks scattered indifferent vertices. *Mitigation:* connected frustrated selector seeds from frustrated vertex and grows connected.
- Plateau collapse** — when $>80\%$ vertices have $|gain| < \text{eps}$, selectors collapse to random. *Mitigation:* degree tiebreaker via `plateau_aware_rank` helper; `frac_zero > 0.8` triggers impact selector in meta-rule.
- Empty pool early** — clustering selector needs ≥ 5 pool entries. *Mitigation:* pool guard falls back to frustrated; clustering only triggers when `len(pool) >= 5`.
- k = N edge case** — `if len(nodes) <= k: return nodes`.

7. **k oscillation** — *Mitigation*: 10-call window + 5-call cooldown + consistent-trend requirement.
-

12. Resume Instructions

To resume in a new chat:

1. Open new chat **in the same project** (Project QLS) so prior conversations remain searchable.
2. Either paste this document or upload to project knowledge (Settings → Project knowledge)).
3. Say: *"Continue the QLS project — see handoff doc. Next step: [whatever]"*.
4. I can also search past conversations directly within the project — ask things like *"find what we decided about the EMA trigger threshold"* or *"pull up the SA cooling literature summary"*.

Useful project-context phrases

- "Continue the k-sweep analysis" — pulls back to step where results need interpretation
- "Implement Phase 2 backend SBM" — picks up at OpenJij integration
- "Run cascade architecture" — picks up sequential SA → SBM → exact
- "Write the results chapter" — pulls all empirical findings together

Minimum Viable Project (if time runs short)

One-flip LS + QLS baseline + Adaptive QLS with all 4 core selectors + neal backend + G-set benchmarks (G1, G11, G14, G22) + evaluation metrics. **This alone is a complete, publishable MSc project.**